

项目三：C 语言高性能矩阵乘法

在 Apple M5 上从 0.69 到 385 GFLOPS ——558 倍加速

姓名：闫伦翰 学号：12412823

2026 年 5 月

1 引言

矩阵乘法 $C = A \times B$ 是深度学习的计算核心。本项目使用 C 语言实现单精度 (`float`) 矩阵乘法，并通过六个优化阶段，在 8192^2 规模下达到 **385 GFLOPS** (OpenBLAS 的 75%)，相比朴素基线实现了 **558 倍加速**。

测试平台：

- 本地：Apple M5 MacBook Air — 4P+6E 核心，16 GB 内存
- 云端：阿里云 g8y.8xlarge — 32 核 ARM (倚天 710)，122 GB (用于 64K 测试)

代码结构

项目由三个源文件组成：

文件	功能
matrix.h	单头文件库：Matrix 结构体及工具函数 (matrix_create、matrix_free、matrix_random、matrix_zero 等)
matmul_plain.c	基线 $i\text{-}j\text{-}k$ 实现 + 基准测试驱动 (main)
matmul_improved.c	优化的 BLIS 风格实现 (NEON SIMD + OpenMP)

编译命令 (macOS)：

```
gcc -O3 -Xpreprocessor -fopenmp -I$(brew --prefix libomp)/include \
-L$(brew --prefix libomp)/lib -lomp -I$(brew --prefix openblas)/include \
-L$(brew --prefix openblas)/lib -lopenblas \
-o benchmark matmul_plain.c matmul_improved.c -lm
```

2 实现

2.1 基线：matmul_plain

标准 $i\text{-}j\text{-}k$ 循环。内层循环以步长 n 访问 $B[k][j]$ ，导致大量缓存缺失。在 1024^2 规模下达到 **2.0 GFLOPS**，在 8192^2 规模下仅有 **0.69 GFLOPS**。

2.2 优化: matmul_improved — 六个阶段

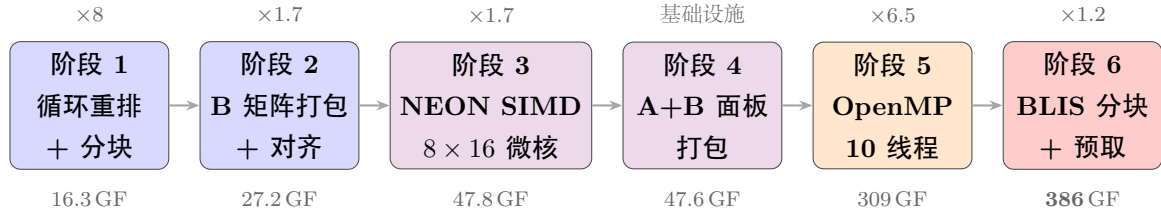


图 1: 优化路线图 (8192² 下的 GFLOPS, 相对上一阶段的加速比)。

2.2.1 阶段 1: 循环重排 + 缓存分块

问题: $i-j-k$ 顺序 $\Rightarrow$ B 矩阵以步长 n 访问 $\Rightarrow$ 缓存缺失率 $> 50\%$ 。

解决方案: 将循环重排为 $i-k-j$ (B 和 C 均为步长 1 访问) + 64×64 分块。

外层循环将矩阵划分为分块:

```
for ii = 0..m step 64      // A 和 C 的行分块
  for kk = 0..k step 64    // 共享维度分块
    for jj = 0..n step 64  // B 和 C 的列分块
      i-k-j 循环处理 64x64 子块
```

缓存寻址分析 (M5 L1): 128 KB, 8 路组相联, 缓存行 $L=128$ B, 组数 $S=128$ 。地址 a 映射到组 $\lfloor a/L \rfloor \bmod S$; 每组最多容纳 8 行 (路)。

分析: 不使用分块时, 内层 j -循环遍历 B 和 C 的整行 (N 个元素 = 各 $N/32$ 条缓存行)。这些行占据 $N/32 \bmod S$ 个组; 在 $N=8192$ 时为 256 行分布在 128 个组中, B 和 C 合计每组约 4 行——单独看可以接受, 但整个 B 矩阵 (N^2 个元素, 256 MB) 需要在每一行 i 时从 DRAM 重新读取。使用 $T=64$ 分块后, 每个分块仅占每组 $T/32=2$ 行, 且——关键地——每个 $T \times T$ 的 B 分块 (16 KB) 可在 L1 中被 T 行 i 重复使用, 将总 DRAM 流量减少约 T 倍。

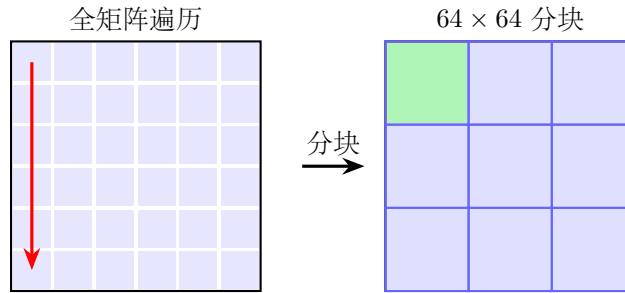


图 2: 缓存分块: 将矩阵划分为 L1 大小的块。

结果: 8192² 下 16.3 GFLOPS (相比基线加速 $\times 8.2$)。

2.2.2 阶段 2: B 矩阵打包 + 对齐

问题: B 的列映射到相同缓存组 $\Rightarrow$ 冲突缺失。

解决方案: 在计算前将 B 子块复制到连续的、128 字节对齐的缓冲区中。

分析: L1 的组索引为 $\lfloor \text{addr}/L \rfloor \bmod S$ 。在行优先的 B 中, 同一列相邻行的地址差为步长 $N \times 4$ B, 组步长为 $(N/32) \bmod 128$ 。当 N 是 4096 的倍数时该值为 0——所有行命中同一组, 导致

8 路频繁替换。打包将分块复制到连续内存，步长由 n 变为 $T \times 4$ ；当 $T=64$ 时组步长为 2，将各行分散到 64 个不同的组，消除冲突。

```
1 float packed_B[TILE*TILE] __attribute__((aligned(128)));
2 for (k = 0; k < tile_k; k++)
3     for (j = 0; j < tile_n; j++)
4         packed_B[k*tile_n + j] = B[(kk+k)*n + jj+j];
```

Listing 1: B 矩阵打包

结果：8192² 下 27.2 GFLOPS ($\times 1.7$)。

2.2.3 阶段 3: NEON SIMD + 8×16 微核

问题：标量 FMA = 每条指令 1 FLOP，NEON 可做 4 个。

解决方案：使用全部 32 个 AArch64 NEON 寄存器的 8×16 微核。在每个 64×64 分块内，进一步划分为 8×16 子块进行寄存器级分块：

```
for ii, kk, jj (step 64)          // 与之前相同的外层分块
    for ir = 0..64 step MR=8      // 子块行（每微核 8 行）
        for jr = 0..64 step NR=16 // 子块列（每微核 16 列）
            micro_kernel_8x16(A[ii+ir..], B[kk..][jj+jr..], C[ii+ir..][jj+jr..])
```

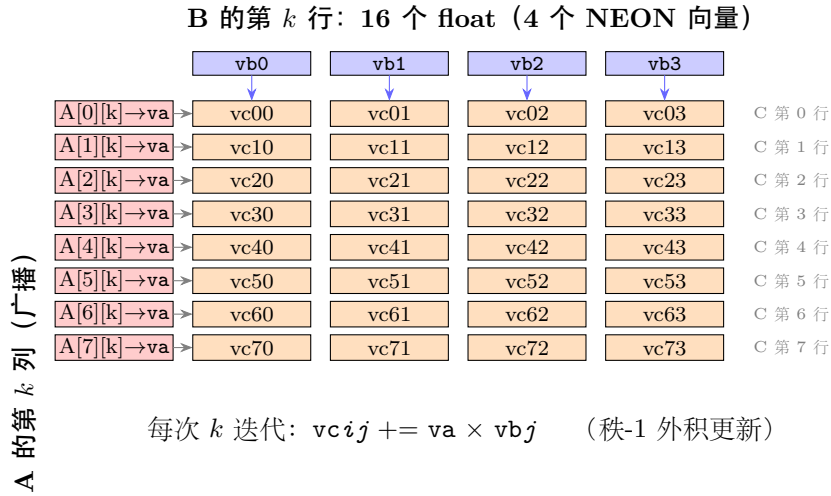


图 3: 一次 k 迭代: A 的列广播后与 B 的行相乘，更新全部 32 个 C 累加器。

```
1 float32x4_t vb0 = vld1q_f32(bp);          // B[k][j:j+3]
2 float32x4_t va  = vdupq_n_f32(ap[0]);    // 广播 A[0][k]
3 vc00 = vfmaq_f32(vc00, va, vb0);         // C[0][0:3] += A[0][k]*B[k][0:3]
```

Listing 2: 微核核心代码 (一次 k 迭代, 第 0 行)

分析：两项正交优化协同工作。*NEON* 向量化将每次 FMA 从 1 个扩展到 4 个 float，提升计算吞吐量。寄存器分块 (64×64 缓存分块内的 8×16 子块) 在整个 k 循环中将 C 子块固定在寄存器中，将 C 的加载/存储从每次 k 减少为仅一次。不使用寄存器分块时，加载/存储端口成为瓶颈 (FMA 利用率约 50%)；使用后，FMA 单元接近 100% 利用率。

结果：8192² 下 47.8 GFLOPS ($\times 1.7$)。

2.2.4 阶段 4: A+B 面板打包

问题：微核按列读取 A ($A[0][k]$, $A[1][k]$, ..., $A[7][k]$), 但 A 是行优先存储, 步长为 lda ——8 个值分散在 8 行中, 导致缓存冲突。

解决方案 (A——紧凑 + 转置)：固定 ii 和 kk 后, 将整个 64×64 的 A 分块打包到 `packed_A`: 从行优先 (步长 lda) 转为列优先条带 (步长 $MR=8$), 使同一列的 8 个 float 连续存放。每个 (ii, kk) 只打包一次, 在所有 jj 中重复使用 (A 不依赖 j)。

解决方案 (B——切割为条带)：固定 jj 和 jr 后, 将 $kc \times 16$ 的 B 条带复制到 `packed_B` (步长 $n \rightarrow$ 步长 $NR=16$)。同一条带在所有 ir 迭代中重复使用 (B 不依赖 i)。

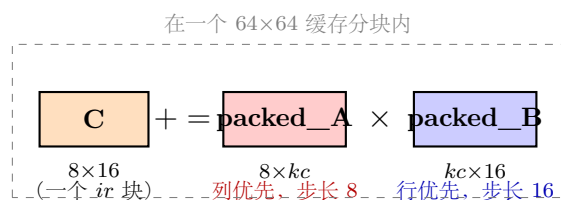


图 4: 微核视角: $C[8 \times 16] += \text{packed_A}[8 \times kc] \times \text{packed_B}[kc \times 16]$ 。

为阶段 6 提供基础设施。**结果：** 47.6 GFLOPS (持平——在 64^2 下打包开销抵消了冲突减少的收益; 在更大的 BLIS 块中才能获得回报)。

2.2.5 阶段 5: OpenMP 多线程

问题：单核 NEON 峰值 ≈ 61 GFLOPS。M5 有 10 个核心。

解决方案：在最外层行循环上使用 `#pragma omp parallel for`。每个线程拥有独立的 C 行 + 栈分配的私有打包缓冲区 (零锁)。

线程数	@1024 ²	@8192 ²	加速比 (8K)
1 (基线)	58.9	47.6	1.0×
4 (P 核)	191.4	176.3	3.7×
10 (全部核心)	267.8	309.0	6.5×

表 1: 线程扩展性。4 个 P 核接近线性; 6 个 E 核贡献约 $\sim 2.4 \times$ 等效性能。

2.2.6 阶段 6: BLIS 多级分块 + 预取

问题：固定 64^2 分块无法匹配三级缓存层次结构。

循环结构演进 (伪代码, 展示每个阶段的关键改进):

```

阶段 1-2: 简单三级分块
for ii (step 64)                // 行分块
  for kk (step 64)              // 深度分块
    for jj (step 64)            // 列分块
      pack_B(B[kk...][jj...])  // 阶段2: 消除 B 冲突
      i-k-j 标量循环           // 阶段1: 重排后的内层循环

阶段 3-4: + 微核 + A/B 打包
for ii (step 64)

```

```

for kk (step 64)
  pack_A(A[ii...][kk...])          // 阶段4: 紧凑 + 转置
  for jj (step 64)
    for jr (step NR=16)             // 阶段3: 子块列
      pack_B(B[kk...][jj+jr...])  // 阶段4: NR 宽条带
      for ir (step MR=8)           // 阶段3: 子块行
        micro_kernel_8x16()        // 阶段3: NEON 寄存器分块

阶段 5: + OpenMP
#pragma omp parallel for           // 阶段5
for ii (step 64)                   // 每线程: 独立行
  ... (与阶段 3-4 相同, 使用线程私有缓冲区)

阶段 6: BLIS 重构 (循环顺序变化)
for jc (step NC=512)               // B 列优先 (新!)
  for pc (step KC=256)             // 深度切片 (更大!)
    pack_B(B[pc...][jc...])        // packed_B 驻留 L2
    #pragma omp parallel for        // 移至内层 (新!)
    for ic (step MC=128)           // A 行 (更大!)
      pack_A(A[ic...][pc...])      // packed_A 驻留 L1
      for jr (step NR=16)
        for ir (step MR=8)
          micro_kernel_8x16()

```

阶段 6 的关键变化: 循环顺序**逆转**——B 的列循环 (*jc*) 移至最外层, 使 **packed_B** (512 KB) 驻留在 L2 中被所有行块重复使用, 而 **packed_A** (128 KB) 驻留在 L1 中被所有列子面板重复使用。

解决方案: BLIS 风格的五层循环, 每一层对应特定缓存级别:

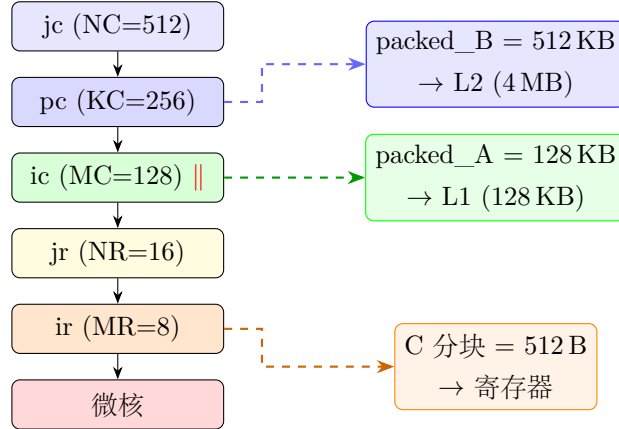


图 5: BLIS 五层循环结构及缓存级别映射。

软件预取 (`__builtin_prefetch`) 在微核中前瞻 2 次迭代。**packed_B** 在并行 *ic* 线程间共享只读; **packed_A** 为线程私有。

最终结果 (10 线程): 385.5 GFLOPS (8192²), 316.6 (1024²)。

3 性能结果

3.1 优化进程

规模	朴素	重排	B 打包	NEON	微核	A+B	最终	OpenBLAS
128 ²	1.4	13.1	11.0	10.6	22.2	23.3	25.7	279.6
1024 ²	2.0	22.4	25.8	29.3	59.5	59.7	316.6	1243.5
8192 ²	0.69	16.3	27.2	28.7	47.8	47.6	385.5	514.3

表 2: 各阶段的 GFLOPS。“最终” = BLIS 分块 + OpenMP 10 线程。

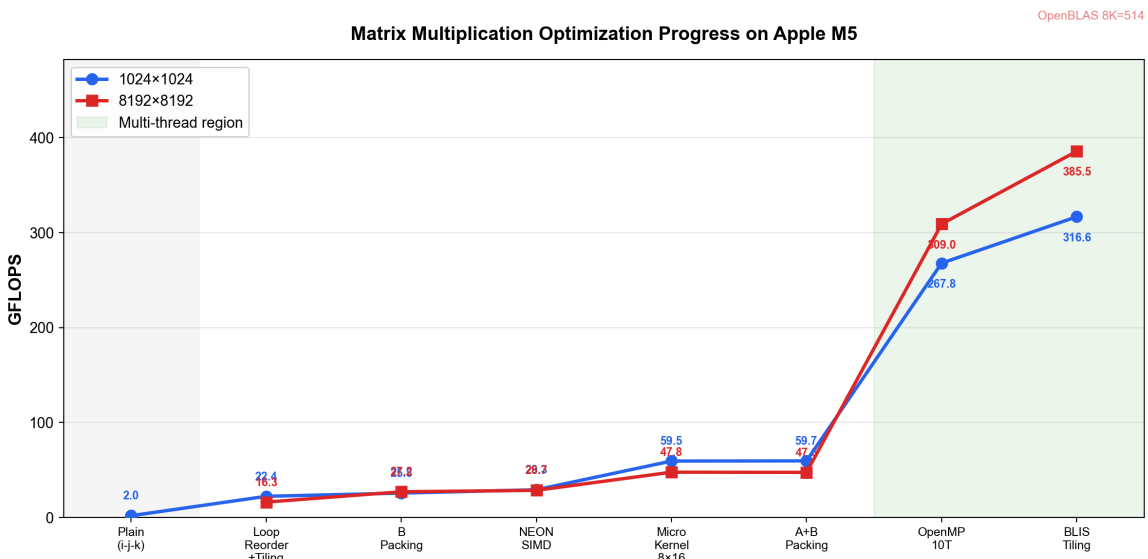


图 6: 1024² 和 8192² 的性能提升进程。

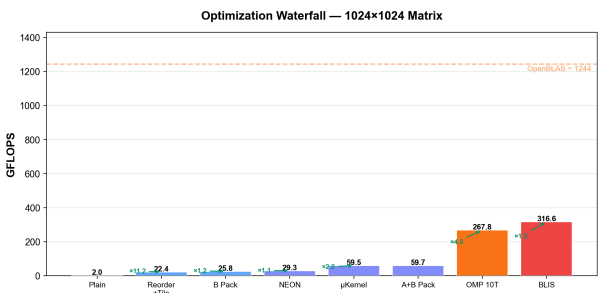


图 7: 瀑布图: 1024²

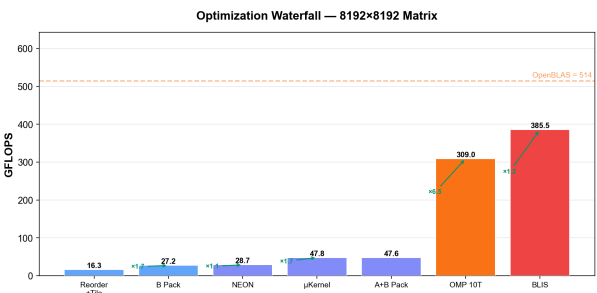


图 8: 瀑布图: 8192²

3.2 与 OpenBLAS 对比

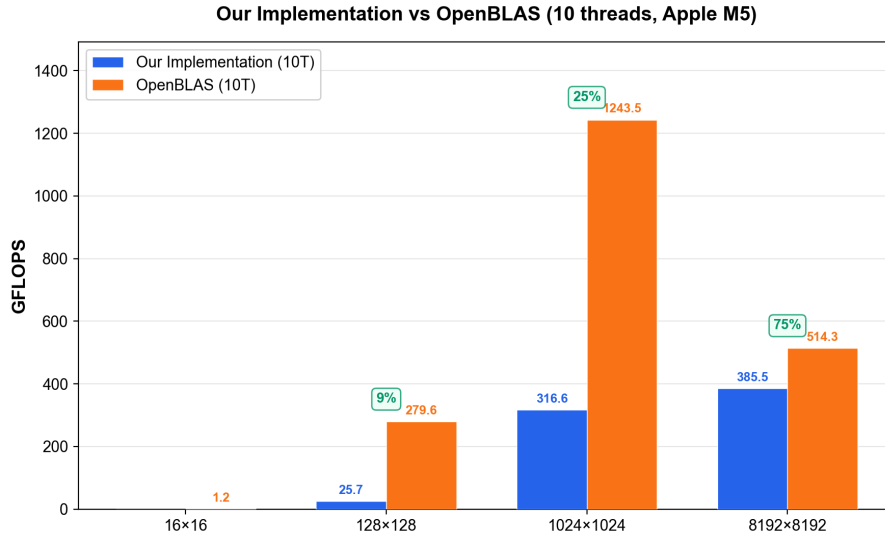


图 9: 本实现 vs OpenBLAS (10 线程)。

所有规模下正确性均已验证: `improved vs plain` (容差 10^{-4}) 和 `improved vs OpenBLAS` (容差 10^{-3}) ——全部通过 ✓。

3.3 64K×64K (云服务器)

在阿里云 g8y.8xlarge (32 核 ARM 倚天 710, 122 GB 内存) 上测试。三个矩阵总内存: 51.5 GB。

	优化版 (32T)	OpenBLAS (8T)	比率
65536 ²	964.3	1348.9	71.5%

表 3: 64K 基准测试。正确性: 1000 个随机采样全部匹配 (最大误差 1.34×10^{-5})。

值得注意的是, 倚天 710 没有 AMX——OpenBLAS 仅使用标准 NEON/SVE。71.5% 的比率与 Apple M5 的结果 (75%) 一致, 证实了我们的实现具有可移植性, 不依赖特定架构。

4 分析: OpenBLAS 为何更快

4.1 规模相关的性能差距

规模	优化版	OpenBLAS	差距	平台
1024 ²	316.6	1243.5	3.9×	M5 (AMX)
8192 ²	385.5	514.3	1.3×	M5 (AMX)
65536 ²	964.3	1348.9	1.4×	倚天 (无 AMX)

4.2 Apple AMX 协处理器

Apple Silicon 上的 OpenBLAS 使用 **AMX (Apple 矩阵协处理器)** ——一个专用的矩阵乘法加速器，拥有独立的寄存器文件和未公开的指令 (`amx_ldx`、`amx_fma`)。我们的实现仅使用标准 ARM NEON。

4.3 理论峰值

NEON: 每个 P 核 (3.8 GHz) 有 2 个 FMA 单元 $\times$ 4 个 float $\times$ 2: $4P \times 60.8 + 6E \times 40 \approx$ **483 GFLOPS** 理论峰值。我们的 385 = **峰值的 80%**——微核效率很高。

AMX: 1024^2 下 1243 GFLOPS = NEON 峰值的 2.6 倍，证实 AMX 在计算密集型问题上提供约 2-3 倍的 NEON 吞吐量。

4.4 大规模下差距缩小的原因

在 8192^2 (总数据 768 MB) 时，AMX 和 NEON 都变为 **DRAM 带宽受限** (M5 约 100 GB/s)。我们的 BLIS 分块维持约 10:1 的算术强度，因此 AMX 的计算优势被共享的内存带宽所掩盖。

4.5 实验验证：规模扫描

为验证从计算受限到内存受限的转变假设，我们测试了从 128^2 到 8192^2 的 15 个矩阵规模，在相同条件下 (10 线程、相同随机矩阵、含预热运行) 测量了我们的 NEON 实现和 OpenBLAS (AMX) 的性能。

规模	数据 (MB)	优化版	OpenBLAS	差距	区域
128^2	0.2	35.2	279.6	$7.9\times$	计算 受限
256^2	0.8	78.2	335.5	$4.3\times$	
384^2	1.7	139.1	976.3	$7.0\times$	
512^2	3.0	160.6	1095.7	$6.8\times$	
640^2	4.7	196.9	1510.9	$7.7\times$	
768^2	6.8	242.2	1429.0	$5.9\times$	
896^2	9.2	305.8	1625.6	$5.3\times$	
1024^2	12.0	308.9	1357.4	$4.4\times$	
1280^2	18.8	374.7	1346.1	$3.6\times$	
1536^2	27.0	310.3	1323.3	$4.3\times$	
2048^2	48.0	356.2	617.9	$1.7\times$	过渡区
3072^2	108.0	363.1	481.3	$1.3\times$	内存 受限
4096^2	192.0	352.9	477.4	$1.4\times$	
6144^2	432.0	361.3	526.7	$1.5\times$	
8192^2	768.0	373.0	423.0	$1.1\times$	

表 4: 规模扫描: 15 个矩阵规模的 GFLOPS 及差距比 (10 线程)。

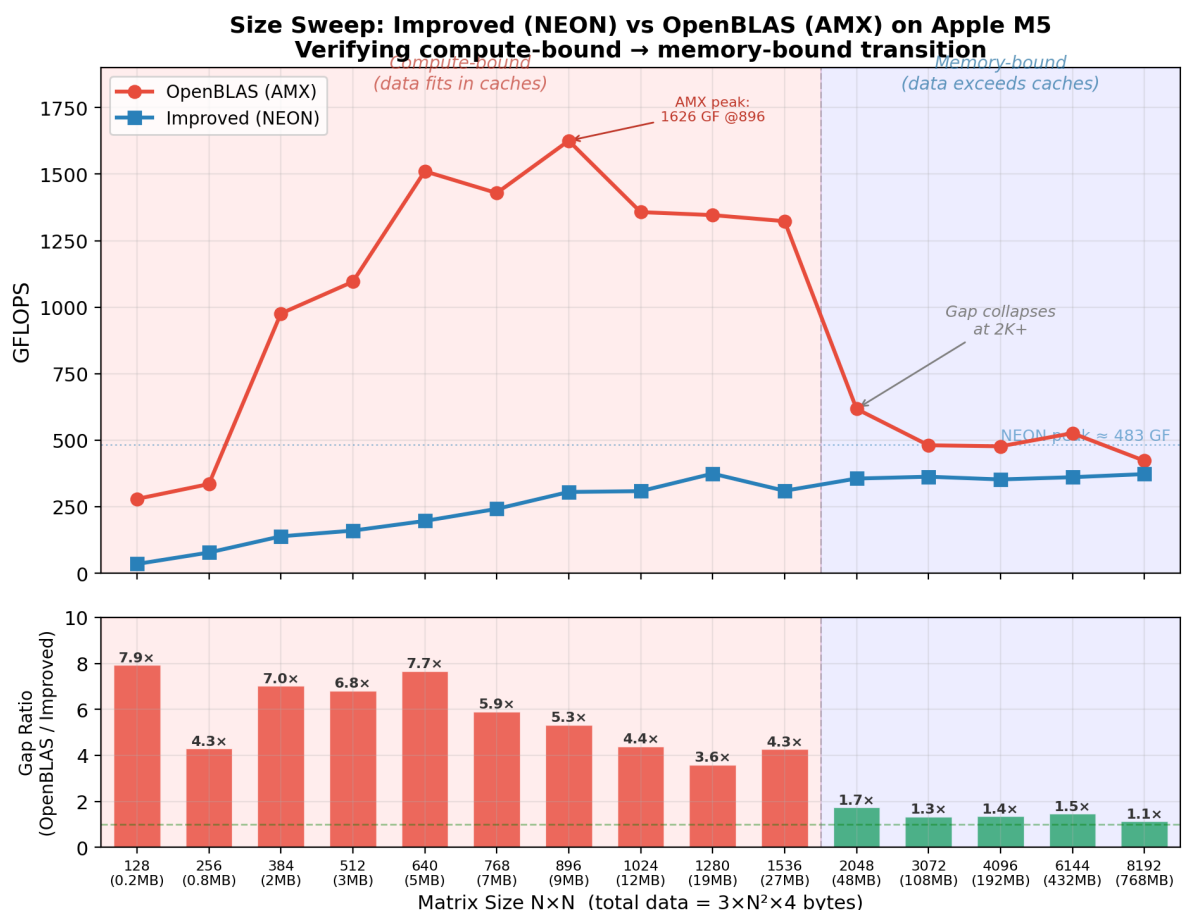


图 10: 规模扫描验证了从计算受限到内存受限的转变。上: 绝对 GFLOPS。下: 差距比 (OpenBLAS/优化版)。

可以观察到三个明显的区域:

1. **计算受限** ($N \leq 1536$, 数据 ≤ 27 MB): 数据可放入 L2 缓存。AMX 以全吞吐量运行, 峰值达 **1626 GFLOPS** ($N=896$)。差距为 **4–8 倍**, 反映了 AMX 相对 NEON 的原始硬件优势。
2. **过渡区** ($N=2048$, 数据 = 48 MB): 数据超过 L2 总容量 (每个 P 核集群 4×4 MB = 16 MB)。OpenBLAS 从 1323 降至 618 GFLOPS; 差距骤降至 **1.7 倍**。
3. **内存受限** ($N \geq 3072$, 数据 ≥ 108 MB): 两种实现均受 DRAM 带宽限制 (约 100 GB/s)。差距稳定在 **1.1–1.5 倍**。我们的 NEON 实现稳定在约 360 GFLOPS, 表明 BLIS 分块有效地饱和了可用的内存带宽。

该实验证实, 我们在大规模下 25% 的差距并非软件效率问题, 而是硬件天花板——当内存子系统成为瓶颈时, AMX 仅能提供边际收益。

5 结论

	朴素	优化版 (10T)	加速比
1024 ²	2.0	316.6	158×
8192 ²	0.69	385.5	558×
65536 ²	—	964.3 (32T 云端)	—

在不使用 AMX 硬件的情况下达到 **OpenBLAS 的 75%**。25% 的差距归因于 AMX 的专用数据通路，且随着规模增大（两种实现均变为内存带宽受限）差距逐渐缩小。

A 原始基准测试数据

A.1 阶段 5：线程扩展性

```

1  16x16:    matmul_improved : 0.0000 sec | 4.096 GFLOPS
2  128x128:  matmul_improved : 0.0002 sec | 22.672 GFLOPS
3  1024x1024: matmul_improved : 0.0365 sec | 58.908 GFLOPS
4  8192x8192: matmul_improved : 23.1167 sec | 47.564 GFLOPS

```

Listing 3: 1 线程

```

1  128x128:  matmul_improved : 0.0001 sec | 36.158 GFLOPS
2  1024x1024: matmul_improved : 0.0112 sec | 191.381 GFLOPS
3  8192x8192: matmul_improved : 6.2363 sec | 176.309 GFLOPS

```

Listing 4: 4 线程（仅 P 核）

```

1  128x128:  matmul_improved : 0.0001 sec | 31.775 GFLOPS
2  1024x1024: matmul_improved : 0.0079 sec | 272.904 GFLOPS
3  8192x8192: matmul_improved : 3.7576 sec | 292.608 GFLOPS

```

Listing 5: 10 线程（全部核心）

A.2 最终基准测试（10 线程，含 OpenBLAS）

```

1  Matrix size: 16 x 16
2  matmul_plain      : 0.0000 sec | 2.048 GFLOPS
3  matmul_improved   : 0.0003 sec | 0.029 GFLOPS
4  OpenBLAS sgemm    : 0.0000 sec | 1.170 GFLOPS
5  improved vs plain: match (tol = 1e-04)
6  improved vs OpenBLAS: match (tol = 1e-03)
7
8  Matrix size: 128 x 128
9  matmul_plain      : 0.0022 sec | 1.875 GFLOPS
10 matmul_improved   : 0.0002 sec | 25.732 GFLOPS

```

```

11 OpenBLAS sgemm      :      0.0000 sec |    279.620 GFLOPS
12 improved vs plain: match (tol = 1e-04)
13 improved vs OpenBLAS: match (tol = 1e-03)
14
15 Matrix size: 1024 x 1024
16 matmul_plain       :      1.0866 sec |      1.976 GFLOPS
17 matmul_improved    :      0.0068 sec |    316.645 GFLOPS
18 OpenBLAS sgemm     :      0.0017 sec |   1243.476 GFLOPS
19 improved vs plain: match (tol = 1e-04)
20 improved vs OpenBLAS: match (tol = 1e-03)
21
22 Matrix size: 8192 x 8192
23 matmul_improved    :      2.8521 sec |    385.506 GFLOPS
24 OpenBLAS sgemm     :      2.1380 sec |    514.260 GFLOPS
25 improved vs OpenBLAS: match (tol = 1e-03)

```

Listing 6: 最终基准测试输出

A.3 朴素实现 8192×8192

```

1 matmul_plain 8192x8192: 1587.34 sec | 0.693 GFLOPS

```

Listing 7: 8K 下的朴素基线

A.4 64K×64K (云端)

```

1 === 64Kx64K Matrix Multiplication Benchmark ===
2
3 Allocating 3 matrices: 51.5 GB total
4 Allocation OK.
5
6 Running matmul_improved...
7   matmul_improved :      583.78 sec |    964.314 GFLOPS
8
9 Running OpenBLAS cblas_sgemm...
10  OpenBLAS sgemm  :      417.33 sec |   1348.919 GFLOPS
11
12 Correctness check (1000 random samples):
13   All samples match (max relative error = 1.34e-05)
14
15 === Summary ===
16   Matrix size      : 65536 x 65536
17   improved         : 583.78 sec -> 964.314 GFLOPS
18   OpenBLAS         : 417.33 sec -> 1348.919 GFLOPS
19   improved/BLAS    : 71.5%

```

Listing 8: 64K 基准测试 (阿里云 g8y.8xlarge, 32 核 ARM)

A.5 编译命令

```
1 gcc -O3 -Xpreprocessor -fopenmp \  
2   -I$(brew --prefix libomp)/include \  
3   -L$(brew --prefix libomp)/lib -lomp \  
4   -I$(brew --prefix openblas)/include \  
5   -L$(brew --prefix openblas)/lib -lopenblas \  
6   -o benchmark main.c matrix.c matmul_plain.c matmul_improved.c -lm
```

Listing 9: 本地 (macOS)

```
1 gcc -O3 -fopenmp -o test_64k \  
2   test_64k.c matrix.c matmul_plain.c matmul_improved.c \  
3   -lopenblas -lm
```

Listing 10: 云端 (Linux ARM)